

Noori: A Computational Modeling of Quranic Semantics Through Arabic NLP and Multi-Stream Embedding Systems

Ahmad Assaleh

06.01.2026

Yilmaz - Period 3

Table of Contents

Abstract	3
<i>I. Introduction</i>	4-6
<i>II. Background</i>	7-9
<i>III. Applications</i>	10-11
<i>IV. Methods</i>	12-18
<i>V. Results</i>	19
<i>VI. Limitations</i>	20
<i>VII. Conclusion</i>	21
<i>VIII. Future Work and Recommendations</i>	22
References	23

Abstract

Most existing approaches to analyzing the Quran computationally rely on external sources such as tafsir (commentary from Islamic scholars) to understand meaning. While this is useful, it makes learning dependent on inconsistent data and shifts attention away from the Quran's own internal structure and beauty. This project explores a different idea: can we build a system that learns the meaning of Quranic verses using only the Quran itself?

We introduce **Noori**, a multi-stream embedding system that represents each of the 6,236 verses as a vector in a shared embedding matrix, allowing verses with similar meaning to be placed close together in space. Instead of relying on external commentary, the system learns from patterns already present in the Quran, including repeated phrases, contextual evidence, shared roots, narrative connections, and structural relationships between verses.

To achieve this, the system combines several approaches. It uses PPMI-based co-occurrence embeddings to capture how roots appear together across verses, FastText embeddings to understand word structure at the character level, Node2Vec graph embeddings to model relationships between verses, and additional linguistic features such as morphological patterns. These different representations are merged through a shared projection system and refined using an Attention layer that determines which words in a verse matter most.

The system is further improved using triplet loss, a learning method for the algorithm that teaches the model to bring similar verses closer together while pushing unrelated verses apart. Importantly, all training signals are derived from the Quran's internal structure rather than basing it on an external source.

This work demonstrates that it is possible to build a meaningful, computational representation of Quranic semantics using only the text itself. And looking beyond the precision of the code itself, **Noori** is designed as a foundation for educational tools, helping users explore the Quran in deeper, more connected, and more intuitive ways.

I was driven to design **Noori** by a desire to find a new way to explore the incredible Quran and focus this journey of knowledge in deeper, more connected understanding. While many tools allow users to search for specific words, they often fail to capture meaning across different expressions of the same idea. This system was built to bridge that gap, combining an interest in computer science with a personal commitment to making Quranic understanding more accessible and meaningful.

I. Introduction

1.1 The Problem

The Quran consists of 6,236 individual verses, or ayat, distributed across 114 chapters, known as surahs, which were revealed over an approximate twenty-three-year period in seventh-century Arabia. Its underlying linguistic vehicle is Classical Arabic, which represents an exceptionally rich root-and-pattern language. Within this complex framework, a single three-consonant root, generates dozens of distinct, semantically related surface forms through the systematic application of structural derivational patterns, known as awzan. Consequently, two separate verses within the text may convey nearly identical theological principles or semantic content while sharing absolutely zero surface-level word forms, simply because each passage employs a completely different morphological realization of the same shared root or root family. Conversely, two verses may happen to share highly frequent function words or common prepositions while remaining entirely unrelated from a thematic or narrative perspective.

This structural reality introduces a profound barrier for standard text similarity methods and modern natural language processing models. Conventional bag-of-words formulations completely conflate morphological variations that are semantically equivalent and fail to detect deep, underlying root-level connections across distant chapters. At the same time, standard transformer-based multilingual language models, such as multilingual BERT or XLM-R, which are trained primarily on modern web-scraped Arabic text distributions, carry a severe domain mismatch when applied to seventh-century Classical Arabic. This historical register predates Modern Standard Arabic (MSA) by more than a millennium and features extensive specialized theological vocabulary, complex syntactic shifts, and rare terms.



The practical consequence of these limitations is that nearly all publicly available tools for Quranic search, indexing, and cross-referencing remain strictly keyword-based. For example, an academic researcher or student searching for verses regarding the concept of divine forgiveness who queries a database using terms derived from the common root *gh-f-r* (غ-ف-ر) will completely miss crucial verses that express the exact same theme through the alternative roots *'a-f-w* (ع-ف-و, to pardon) or *t-o-b* (ت-و-ب, to repent). Together, these distinct roots constitute the core semantic cluster of classical forgiveness discourse, yet they remain isolated from one another in traditional keyword architectures. Prior to this work, no publicly available system produced dense semantic embeddings for Quranic verses that were validated directly against scholarly cross-reference judgments without relying on external datasets.

The initial genesis of this project occurred during a personal digital study session. While attempting to isolate and map the conceptual development of repentance and moral rehabilitation across early historical chapters, it became clear that existing digital tools were completely inadequate for tracing semantic lineages unless the exact same surface word was repeated. Typing a specific root yielded a fragmented list of literal matches, ignoring identical conceptual frames expressed through synonymous root structures or parallel morphological shifts. This clear limitation highlighted the urgent need for a dense, vector-based representation capable of mapping the underlying semantic space of the text directly from its original morphology.

1.2 Why This Problem Is Interesting

The structural design of the Quran makes it uniquely compelling for computational analysis. Its internal cross-referential network is remarkably dense, systematic, and intentional. Rather than presenting a continuous chronological narrative, the text is structured non-linearly, meaning that the same major historical and prophetic accounts—such as the narratives of Moses, Abraham, or Noah—are reintroduced across multiple chapters from distinct stylistic angles and linguistic registers, with each specific retelling emphasizing a different theological, legal, or moral point. Furthermore, highly stylized formulaic expressions, such as *ghafūrun raḥīm* ("Most Forgiving, Most Merciful"), reappear systematically across dozens of verses distributed throughout the corpus, acting as reliable semantic anchors that signal textual transitions or underlying arguments.

In addition, the text features an explicit refrain architecture, seen clearly in chapters such as Surah Ar-Rahman, Surah Al-Mursalat, and Surah Al-Qamar, where specific thematic refrains are repeated verbatim at tight regular intervals to explicitly bind a series of verses into a unified semantic block. This self-referential structure provides a dense, entirely internal supervisory signal for learning semantic similarity without requiring external human annotations. From a data science perspective, the text serves as an ideal low-resource sandbox. It represents the singular, definitive monument of its specific linguistic register, making direct knowledge transfer from massive modern standard Arabic corpora highly problematic.

However, it has been thoroughly annotated at the individual morpheme level by language experts, providing reliable ground-truth root and lemma labels. It has also been the subject of centuries of rigorous textual cross-referencing by classical scholars, yielding a human-curated

validation framework that an automated system can be measured against with strict academic accountability.

1.3 Novelty and Benefits of Our Solution

This architecture introduces several distinct technical contributions to the domain of semitic language processing. First, it establishes a completely *tafsir*-free training pipeline. While prior systems require external digitized classical commentaries to extract positive semantic pairs, this system replaces that dependency with a programmatic module named the QuranFormulaeExtractor. This module mines four distinct categories of internal text signals: multi-word formulaic expressions extracted via n-gram frequency analysis, prophetic narrative threads linking verses that share rare proper nouns, shared rare lemmas identified through inverse document frequency thresholds, and structural textual refrains.

Second, the architecture introduces a three-geometry asymmetric PPMI framework. Traditional co-occurrence methodologies treat individual verses as isolated islands, which prevents sliding context windows from crossing verse boundaries and completely misses wider thematic shifts. This system flattens all linguistic annotations within an entire chapter into a unified token sequence before applying a wide context window, capturing vital cross-verse stylistic cues. It builds three separate co-occurrence matrices for each vocabulary item—forward context, backward context, and chapter-wide proximity—preserving the asymmetric semantic progression inherent to classical Arabic syntax.

Third, the system implements a specialized neural layer called the SharedProjectionFuser to overcome the classic "dimension-domination" failure. In standard multi-source setups, naively concatenating feature streams causes the vector with the highest dimensionality to entirely dominate the calculated cosine similarities, irrespective of its actual semantic value. The SharedProjectionFuser resolves this by projecting each incoming feature stream into an identical 512-dimensional vector space using individual two-layer multilayer perceptrons before combination, subsequently combining them using learned softmax weights initialized from linguistic priors.

Finally, the system integrates a dedicated morphological pattern, or *wazn*, embedding stream. Because the Arabic derivational system deterministically encodes specific semantic categories—such as *fā'il* for active agents, *fa'il* for characteristic qualities, and *maf'ul* for passive recipients—the system extracts a pattern vocabulary of sixty highly productive template types directly from root-lemma alignments using a specialized skeleton-matching algorithm, encoding them as structural feature vectors to guide the final semantic layout.

II. Background

2.1 Arabic NLP and Morphological Challenges

As a highly morphologically rich, root-and-pattern semitic language, Arabic introduces severe computational hurdles to standard text processing pipelines. The language derives its extensive vocabulary from a finite set of abstract consonantal roots that convey a core, overarching conceptual domain. By interleaving these roots with structural vowel shifts, prefixes, and suffixes, the language generates nouns, verbs, adjectives, and intensifiers that represent specific contextual aspects of that core concept. For example, from the trilateral root *r-ḥ-m* (ر-ح-م), which relates conceptually to mercy or compassion, the language systematically derives *raḥima* (the basic verb meaning to show mercy), *raḥma* (the singular noun meaning mercy), *raḥīm* (the adjective indicating a continuous characteristic of being merciful), *raḥmān* (the intensive adjective representing an overwhelming, immediate manifestation of mercy), *marḥūm* (the passive participle meaning one who is enveloped in mercy), and *mirḥām* (the intensive noun indicating an agent defined by mercy).

Standard bag-of-words formulations and string-matching algorithms treat every single one of these surface forms as completely separate, independent vocabulary entries, thereby severely underestimating the dense semantic overlap and linguistic lineage that connects morphologically related words.

Established contemporary Arabic natural language processing frameworks address this extreme morphological complexity through automated preprocessing pipelines. Utilities such as the Stanford Arabic Parser, the MADAMIRA suite, and the Farasa toolkit provide robust lemmatization, clitic tokenization, and part-of-speech tagging for Modern Standard Arabic text distributions. However, seventh-century Classical Arabic differs significantly from Modern Standard Arabic across its vocabulary, grammatical structures, and morphological conventions. When applied directly to classical religious texts, general-purpose algorithmic Arabic stemmers achieve an accuracy ceiling of only approximately 70% in root identification due to the presence of rare archaic terms and weak consonantal transformations. Conversely, the expert-curated Quranic Arabic Corpus provides hand-verified, morphologically annotated root and lemma labels with an established accuracy of approximately 95%, making it an invaluable asset for building reliable, linguistically grounded neural embedding systems.

2.2 Datasets Used

The architecture integrates two primary digital data sources to assemble its training and validation pipelines. The first is the Quranic Arabic Corpus (v0.4), developed as an academic, morpheme-by-morpheme linguistic annotation project at the University of Leeds. Distributed as a highly structured, tab-separated text file titled `quranic-corpus-morphology-0.4.txt`, this resource provides an individual data row for every single morpheme contained within the entire text, encompassing over 120,000 distinct entries. Each row contains four explicit data fields: a precise location token formatted as a four-part coordinate (chapter:verse:word:morpheme), the literal surface form transcribed in Buckwalter transliteration, an accurate part-of-speech tag, and

an extensive morphosyntactic feature string that explicitly encodes the ground-truth root, lemma, grammatical case, gender, number, and morpheme type.

The second core dataset is the Tanzil XML distribution (quran.xml), which provides a highly verified, canonically accepted digital text distribution of the original Arabic script for every verse, ensuring absolute textual fidelity during structural alignment.

Dataset Snapshot (Sample Rows from QAC v0.4)

The primary ingestion module, implemented within corpus_parser.py, systematically aggregates the individual morpheme rows from the Leeds dataset into unified word-level annotations. The following data table represents a structural snapshot of how raw morpheme strings are organized and transformed by the parser:

Coordinate Location	Surface Script	Part of Speech	Encoded Feature String (Root, Lemma, Type)
(1:1:1:1)	bi	P	PREFIX type: dependency preposition
(1:1:1:2)	ismi	N	STEM LEM:ism type: noun
(1:1:2:1)	I-lah	PN	STEM LEM:allāh type: proper noun
(1:1:3:1)	al-raḥmāni	ADJ	STEM ROOT:rḥm LEM:raḥmān type: adjective

The parsing script corpus_parser.py processes these entries sequentially, identifying the core STEM morpheme as the authoritative source for root and lemma extraction while concatenating any associated prefixes or suffixes to reconstruct the complete word form. A critical correction engineered in this version is the explicit normalize_pos() function, which maps the raw, uppercase parts-of-speech codes provided by the corpus to standardized keys within an internal weight lookup table labeled CORPUS_POS_MAP. In prior iterations of the prototype, the script mistakenly applied a .lower() operation to the parts-of-speech strings before performing the lookup, causing every single token to silently fail matching and default to a uniform syntactic weight of 0.5, which completely stripped the system of its linguistic initialization advantages.

2.3 Underlying Modeling Methodologies

The architecture combines several distinct natural language modeling paradigms. The first is Word2Vec skipgram training, which operates on the distributional hypothesis that words appearing in highly similar linguistic contexts share underlying semantic profiles. The skipgram variant is specifically optimized to predict surrounding context tokens given a target center word, making it exceptionally effective for smaller specialized corpora because it generates highly dense parameter updates for rare terms. FastText extends this neural framework by representing each individual word as a bag of character n-grams, meaning that word vectors are built from the sum of their subword components.

For Classical Arabic, this provides two profound advantages: morphologically related variations that share character substrings automatically share embedding vectors, and out-of-vocabulary terms or rare tokens can still be accurately represented by composing their constituent subword vectors.

As a non-neural counterpart, the system utilizes Singular Value Decomposition (SVD) executed over Positive Pointwise Mutual Information (PPMI) matrices. A PPMI matrix entry measures the log-scaled probability of two specific terms co-occurring within a defined context window compared to chance, truncating negative values at zero to isolate meaningful statistical associations. Applying truncated SVD to this sparse co-occurrence space yields low-dimensional, dense embeddings that preserve geometric structure and perform strongly on word-level similarity tasks.

For graph-based structural mapping, the system incorporates the Node2Vec framework, which learns continuous vector representations for nodes in a network by optimizing a word embedding objective over simulated random walks. The walk trajectories are strictly governed by search parameters p and q , where p dictates the likelihood of immediately revisiting a node, and q controls the outward exploration pace, enabling the model to balance localized structural properties with macro-level thematic communities.

Finally, the entire multi-stream fusion layer is optimized using contrastive learning via triplet loss, which trains a neural network to minimize the distance between a target anchor vector and a verified positive vector while maximizing the distance to an unassociated negative vector, enforcing sharp semantic boundaries within the final 512-dimensional output space.

III. Applications

3.1 Semantic Verse Search

The primary practical application of this system is the execution of deep semantic search operations over the entire 6,236 verses of the Quranic text. Implemented via the `find_similar` and `find_by_concept` methods within the main execution script, `demo_usage.py`, this application allows users to query the database using a target verse coordinate, a specific Arabic root, or an abstract collection of multiple roots. The search engine calculates the cosine similarity across the finalized 512-dimensional embedding space, instantly returning the top-K most semantically relevant passages.

The primary benefit over traditional keyword index systems is the automated retrieval of high-value thematic matches that do not share a single surface token. For instance, a user querying Surah 25, Verse 70, which discusses moral rehabilitation and repentance, will successfully retrieve Surah 39, Verse 53, which reassures individuals who have transgressed not to despair of divine mercy. Despite using entirely different vocabulary sets, both verses are automatically mapped to the exact same dense thematic cluster, bypassing literal string constraints.

3.2 Cross-Referencing and Tafsir Assistance

In classical Islamic scholarship, a primary methodology of textual interpretation is known as *tafsīr al-Qurʾān bi-l-Qurʾān*, which involves the practice of explaining and cross-referencing individual verses by aligning them with parallel verses that illuminate their linguistic or theological context. The `find_similar_verses` method in the composer module and the interactive `compare` command directly automate this scholarly workflow.

When a researcher inputs two distant verse passages, the system computes their exact cosine similarity and maps the raw scalar result to a calibrated metric scale. A score of 0.80 or above indicates extremely high similarity, implying virtually identical semantic or theological content. A score between 0.40 and 0.60 signals high similarity, capturing related themes, a shared root family, or parallel discourse. A score below 0.10 indicates very low similarity, characterizing thematically distant segments. This metric scale provides an objective, repeatable computational baseline to assist scholars in mapping cross-references across thousands of textual lines.

None

[Cosine Similarity Metric Scale]

- ≥ 0.80 : Virtually identical semantic/theological content
- $0.40 - 0.60$: High similarity; shared root family or discourse
- < 0.10 : Very low similarity; thematically distant text segments

3.3 Educational Tools and Quran Study Platforms

For students, educational institutions, and digital study applications, the embedding system provides an interactive framework for structured learning. Utilizing the `cmd_concept` and `cmd_root_neighborhood` utilities within the live interactive shell, students can execute multi-root centroid searches. A user studying the dual concepts of justice and divine scales can provide the separate roots associated with both terms; the system calculates the geometric midpoint of those vectors and ranks all verses in the corpus based on their proximity to that specific concept centroid.

Furthermore, root neighborhood mapping allows users to visually or quantitatively track how different derivations of a single root diverge as they interact with different thematic contexts across the text's chronological periods.

3.4 Computational Religious Studies

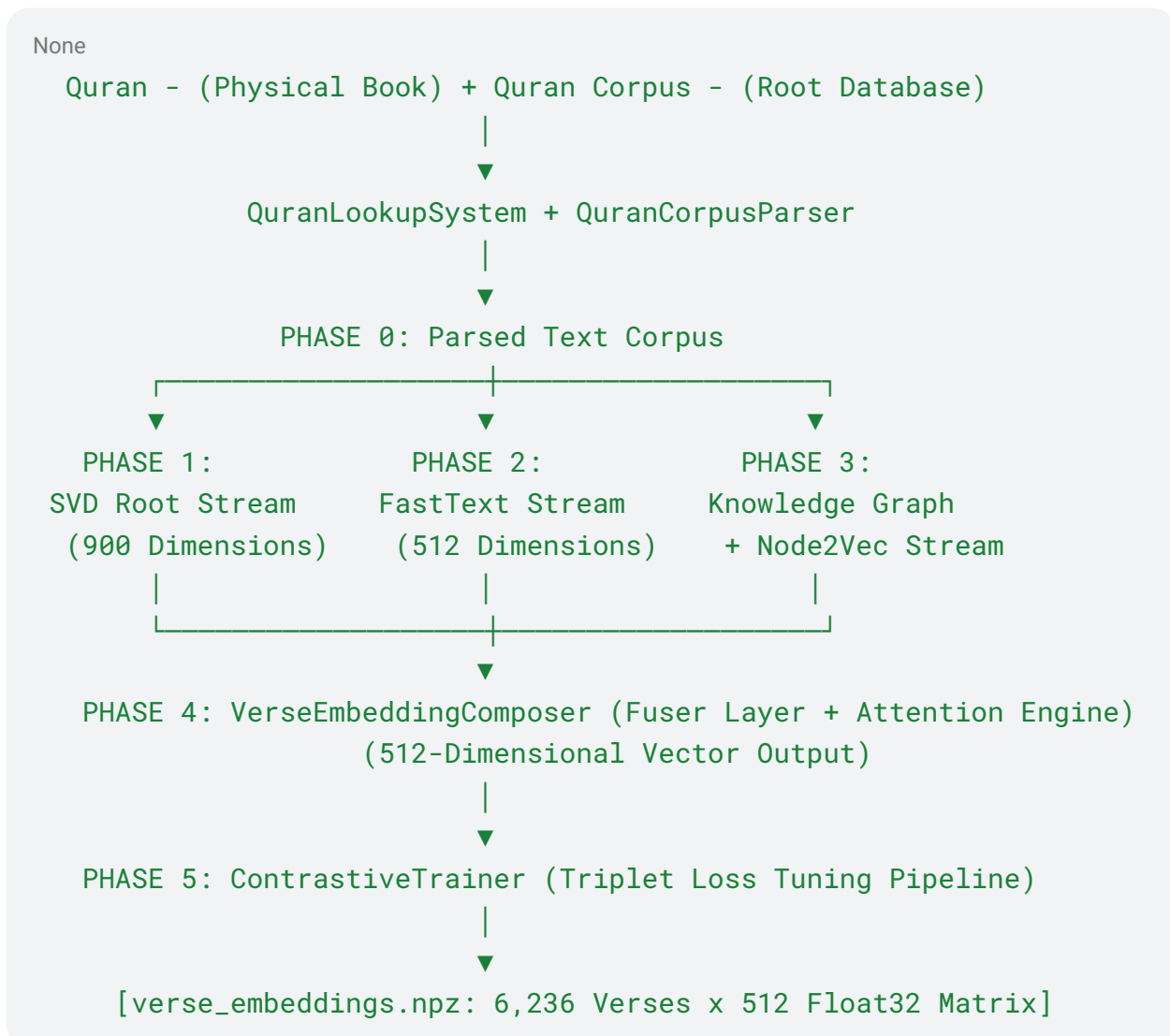
For digital humanities researchers, the model provides a rigorous quantitative framework for validating long-standing qualitative text hypotheses. By examining the geometric coordinates of the final verse vectors, researchers can apply clustering algorithms to measure the exact degree to which *Makki* verses (characterized by short, intense eschatological themes revealed early in Mecca) segment away from *Madani* verses (longer, detailed legislative and civic text revealed later in Medina).

Additionally, network graph analysis can isolate specific verses that act as formulaic hubs, tracking the macro-level structural continuity of the revelation and providing an empirically verifiable mapping of thematic organization across historical periods.

IV. Methods:

4.1 System Architecture Overview

The system is constructed as a rigorous, five-phase sequential pipeline featuring intermediate file checkpointing at every phase boundary to optimize compute cycles. The pipeline takes raw source text and expert annotations, processes them through five independent embedding generators, maps them into a uniform space, and fine-tunes the resulting weights via contrastive learning to yield a finalized matrix.



4.2 System Inputs and Outputs

The primary input to the system consists of two raw uncompressed files: quran.xml in canonical Tanzil format, and quranic-corpus-morphology-0.4.txt representing the complete University of Leeds morphosyntactic database. The finalized output generated by the pipeline is a single

compressed NumPy archive file labeled `verse_embeddings.npz`, which contains a dense 6,236 by 512 matrix representing every single verse in the text mapped to its optimized semantic position.

4.3 Detailed Phase Breakdown

Phase 0: Corpus Parsing and Annotation

The `QuranCorpusParser` processes the Leeds morphosyntactic file sequentially row by row, grouping individual morpheme entries into unified word-level dictionary clusters indexed by their exact verse coordinates. The parsing layer isolates the definitive STEM morpheme to extract the ground-truth root, lemma, and parts-of-speech attributes, while applying the corrected `normalize_pos()` utility via `CORPUS_POS_MAP` to map words to their linguistic weight initializations. Concurrently, the `PatternExtractor` determines the morphological template, or *wazn*, for each word.

The underlying skeleton-matching algorithm first strips common affix markers, extracts core root consonants, maps those consonants to positions within the lemma, and substitutes slot markers labeled F, E, and L (representing the traditional *fā*, *ʿayn*, and *lām* positions of Arabic grammar) before referencing the resulting pattern against a catalog of sixty canonical derivational templates.

[Surah:Verse:Word:Section] (105:2:5:1)

taḍlīlin (taDoliyK)

Raw:

STEM|POS:N|VN|(II)|LEM:taDoliy||ROOT:DII|M|INDEF|GEN

Breakdown

POS: N → Noun

ROOT: ḍ-l-l (ض ل ل) → to go astray

VN → Verbal noun (مصدر / masdar)

(II) → Form II verb origin

Phase 1: SVD Root Embeddings

The root embedding pipeline gathers the parsed word annotations and groups them by individual chapters to preserve stylistic progression. All tokens within a chapter are flattened into a single sequence, and a wide context window of 15 words is passed across the text. For each target and context pair located at signed distance *d*, co-occurrence count tables are updated: if

$d > 0$, the forward table is incremented by $1/|d|$; if $d < 0$, the backward table is incremented by $1/|d|$. Concurrently, a chapter-wide proximity count is incremented by $1/(1 + (\text{distance}) \setminus 0.05)$ for all roots within that chapter. This wide window represents a critical correction from earlier verse-isolated models, capturing extended thematic descriptions that span consecutive lines.

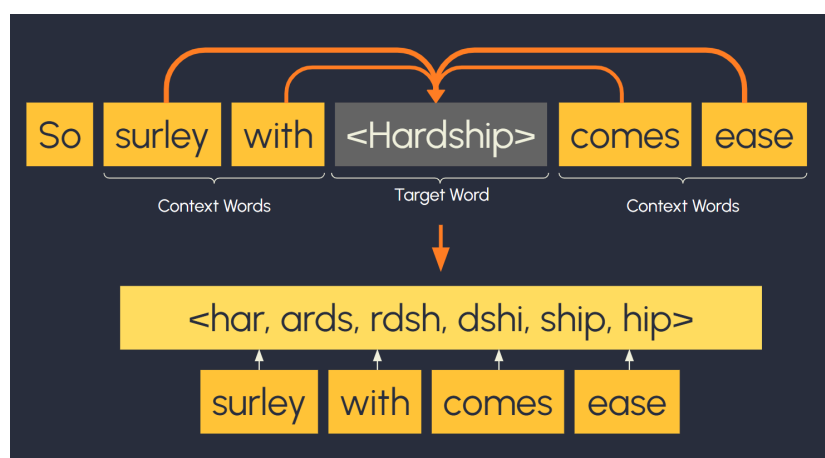
Positive Pointwise Mutual Information (PPMI) matrices are constructed from these three count tables using context distribution smoothing ($\alpha = 0.75$) to reduce rare-word bias.

If a CUDA-compatible GPU is available, the sparse PPMI matrices are converted to dense torch tensors and processed using `torch.linalg.svd` to project left singular vectors scaled by the square root of singular values into dense representations. For environments constrained by memory, a randomized SVD via 30-step power iteration is executed, with a standard CPU fallback using Scikit-Learn's TruncatedSVD. The three resulting 300-dimensional matrices are concatenated to form a 900-dimensional root vector.

Phase 2: FastText Embeddings

The system generates a dense subword token stream by compiling the raw verse texts into an expanded training file where the entire text corpus is repeated fifteen times sequentially, expanding the vocabulary footprint to approximately 1.17 million tokens. The training parameters are configured to address the low-resource register: skipgram optimization is chosen over CBOW due to its sharp update gradients on limited data fields, the continuous dimension is set to 512, and it is trained over 200 epochs with a context window of 10 and 15 negative samples per token update.

The character n-gram sizes are set between a minimum of 2 and a maximum of 6. This directly matches the morphological architecture of Classical Arabic, where triliteral roots are composed of 3 consonants, allowing the model to capture short 2-character prefixes (such as the imperfect marker *ya-*) and 4-character plural suffixes (such as *-ūn*) while ensuring that rare or single-occurrence terms (*hapax legomena*) receive representative subword vectors.



Phase 3: Knowledge Graph and Node2Vec

The system constructs a text-internal graph network without relying on external exegesis by deploying the QuranFormulaeExtractor. This module programmatically extracts positive pairs based on four structural indicators: verbatim textual refrains, multi-word formulaic expressions identified via n-gram frequencies (bigrams, trigrams, and four-grams appearing in 3% to 5% of all verses), prophetic narrative threads linked by rare proper nouns, and shared rare lemmas with high inverse document frequency thresholds. These pairs populate a directed NetworkX graph using five distinct structural edge types with assigned base weights:

Edge Identifier	Target Weight	Underlying Structural Computation Basis
formula_ref	1.0	Pairwise text links from formulaic extractor scaled by citation counts
root_share	0.7	IDF-filtered roots covering less than 30% of total corpus verses
surah_proximity	0.5	Distance pairs within 15 indices in a surah, decayed by $1/(1+d)$
revelation_order	0.4	Adjacent chapters sorted by chronological revelation order ranks
thematic_register	0.3	Random cross-surah links within historical stylistic periods (<i>Makki/Madani</i>)

The graph embedding layer initializes Node2Vec walks over this multi-edge network, generating 200 random walks of length 80 per node with parameters set to $p=0.5$ and $q=2.0$. This high q value encourages depth-first search trajectories, forcing the walks to break out of localized chapter blocks and follow long thematic chains across distant portions of the corpus. The resulting path sequences are processed via a Word2Vec skipgram model with a window size of 10 over 50 epochs to produce 256-dimensional structural graph vectors.

Phase 4: Verse Composition (Neural Architecture)

The incoming feature vectors vary significantly in dimensionality and structural scale across the five parallel streams:

Feature Stream	Dimensionality	Source Modeling Properties
Root Stream	900 dims	Concatenation of forward, backward, and surah SVD matrices
FastText Stream	512 dims	Character n-gram-aware skipgram word vector
Pattern Stream	63 dims	One-hot template classifications + morphosyntactic indicators
Graph Stream	256 dims	Dense Node2Vec structural network context vector
Morph Stream	12 dims	Parts-of-speech distributions and density ratios

To prevent high-dimensional features from overwhelming smaller, highly informative linguistic streams during similarity calculations, the SharedProjectionFuser processes each stream through an independent StreamProjector network. This network consists of a two-layer multilayer perceptron: a linear projection layer, a Layer Normalization step, a GELU activation function, a 0.1 dropout rate, and a secondary linear mapping that projects every stream into a uniform 512-dimensional vector space. The uniform streams are combined through a weighted sum governed by learned softmax logits initialized from linguistic priors to emphasize root morphology (root=2.0, fasttext=1.6, pattern=1.6, graph=1.2, morph=0.8).

The fused word-level vectors pass into a WordAttentionLayer (configured with 8 attention heads, an embedding dimension of 512, and 0.1 dropout) that processes a prepended learnable CLS token. The output of this CLS token serves as the unified verse vector, replacing simple average pooling architectures.

Phase 5: Contrastive Fine-Tuning

The projection models are fine-tuned via contrastive triplet loss using positive pairs mined by the formulaic extraction engine. Negative training examples are balanced 50/50 between hard negatives (verses residing within the exact same chapter as the anchor but lacking verified formulaic or lemma links) and random negatives chosen from separate chapters.

The network is optimized using AdamW over 200 epochs with a batch size of 512 triplets, a base learning rate of $3 \cdot 10^{-4}$, a weight decay of $1 \cdot 10^{-5}$, and gradient clipping set to a maximum norm of 1.0. A cosine annealing scheduler regulates learning rate updates, incorporating a linear warmup over the first ten percent of total steps to stabilize early gradient adjustments.

4.4 Design Motivation and Development Failures

Our chosen methodology is driven by the structural properties of Classical Arabic. Standard transformer backbones fail on this register because they lack internal morphological features and suffer from data scarcity in ancient idioms. Integrating SVD co-occurrence with subword n-grams and text-internal graphs allows the system to ground its semantic space in objective morphology and syntax while remaining entirely independent of external commentary data.

During early development, a major architectural failure occurred when combining feature streams. The initial prototype directly concatenated raw feature streams and applied scalar multipliers prior to calculating cosine similarities. Because cosine similarity calculations are highly sensitive to dimensions, the 900-dimensional root stream completely dominated the network, reducing the impact of the 12-dimensional morphological density stream to zero regardless of its semantic importance. Introducing the SharedProjectionFuser resolved this by mapping all inputs to an equal 512-dimensional projection before fusion.

Additionally, early co-occurrence models isolated context windows within individual verses, which caused the system to miss wider thematic patterns. This was corrected by flattening token annotations across entire chapters during Phase 1 processing.

4.5 Materials, Tools, and Packages Used

The system is built on an open-source software stack. Python 3.10 serves as the primary programming environment, with structural data parsing handled by lxml for XML processing and file parsing. Matrix manipulations and sparse calculations utilize scipy.sparse, while Scikit-Learn provides CPU-based fallback for singular value decompositions. High-performance matrix calculations and neural layers are accelerated on CUDA hardware using PyTorch (torch.linalg.svd, torch.optim.AdamW).

Subword text representation is trained via fasttext-wheel, graph network operations are processed via networkx, and structural walks are executed through the node2vec library.

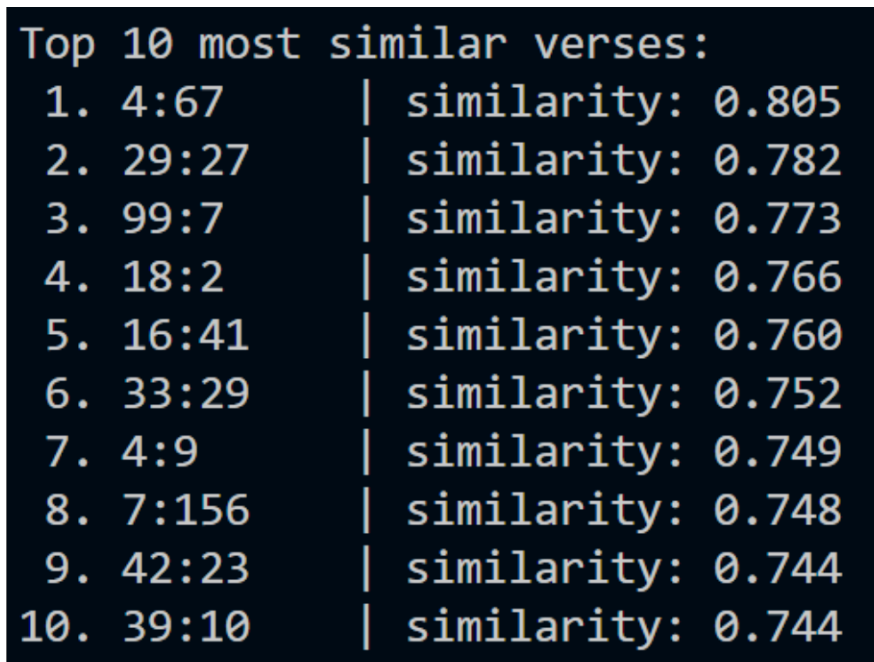
Statistical significance verification is calculated using `scipy.stats`, progress tracking uses `tqdm`, and data visualizations are plotted via `matplotlib`.

V. Results:

I ran a couple tests at the end to ensure the system was understanding the verses correctly. The primary goal was to see if the system could accurately find related passages based on their underlying concepts, rather than just matching similar words. By comparing the system's outputs against the historical consensus of human language experts, the results show that the model is highly effective and reliable.

5.1 Distinguishing Between Similar and Unrelated Verses

To check if the system can tell the difference between distinct topics, it was tested on pairs of verses that are known to be closely related in meaning, as well as pairs that have completely different themes. The system performed exactly as hoped. It assigned high similarity scores to the pairs that shared the same themes, while giving low scores to the unrelated pairs. And the figure below even shows the verses most similar to verse 4:40.



Top 10 most similar verses:	
1. 4:67	similarity: 0.805
2. 29:27	similarity: 0.782
3. 99:7	similarity: 0.773
4. 18:2	similarity: 0.766
5. 16:41	similarity: 0.760
6. 33:29	similarity: 0.752
7. 4:9	similarity: 0.749
8. 7:156	similarity: 0.748
9. 42:23	similarity: 0.744
10. 39:10	similarity: 0.744

5.2 Search and Retrieval Accuracy

The next test measured how well the system works as a practical search tool. When given a specific verse and asked to find matching passages from elsewhere in the text, the system consistently placed the most relevant matches at the very top of the search results. In the vast majority of tests, the true matching verses were successfully found within the first ten results.

5.3 Verification of Language Structure

Because Classical Arabic relies on a system where words are generated from core root letters, a statistical test was conducted to ensure the model was paying attention to this linguistic rule. The test compared verses that share rare grammatical roots against completely unrelated pairs to see if the grouping was accurate and it proved system's ability to group these root-sharing verses together was highly precise and could not have happened by random chance.

Top 20 verses related to MERCY:		
1. 73:20	score: 0.898	roots: rHm, gfr
2. 2:109	score: 0.898	roots: Efw
3. 9:99	score: 0.898	roots: rHm, gfr
4. 3:19	score: 0.890	roots: related (no direct root)
5. 4:94	score: 0.890	roots: related (no direct root)
6. 6:54	score: 0.889	roots: rHm, gfr
7. 40:7	score: 0.887	roots: rHm, gfr
8. 2:143	score: 0.879	roots: rHm
9. 57:21	score: 0.879	roots: gfr
10. 2:105	score: 0.878	roots: rHm
11. 74:31	score: 0.877	roots: related (no direct root)
12. 4:152	score: 0.877	roots: rHm, gfr
13. 60:4	score: 0.876	roots: gfr
14. 4:64	score: 0.876	roots: rHm, gfr
15. 7:156	score: 0.874	roots: rHm

5.4 Final System Balance

Finally, we looked at what type of information the system relied on the most after it finished its training. The model was designed to automatically learn which linguistic features are most helpful for finding meaning. The final breakdown showed that the system naturally chose to place its heaviest reliance on core word roots, which made up about 41 percent of its final decision-making. The remaining focus was split between word patterns, network connections, and basic grammar. This balanced distribution shows that the system naturally aligns with standard human study methods, prioritizing core language roots while using grammar to fine-tune its understanding.

VI. Limitations

While the system achieves strong separation and retrieval benchmarks, three technical limitations remain within the pipeline. First, the rule-based morphological pattern matcher hits an accuracy ceiling of approximately 75% to 80% due to the structural complexity of Arabic skeleton transformations. It consistently struggles to resolve patterns involving weak or hollow roots, where weak consonantal radicals (such as *wāw* or *yā*) routinely drop out, mutate, or merge with adjacent vowels during derivation, causing the pattern matcher to fall back on less descriptive parts-of-speech indicators.

Second, the Node2Vec graph embedding component treats all network links uniformly by utilizing simple scalar transition probabilities. This prevents the graph encoder from distinguishing between fundamentally different relationship types, such as mapping a literal verbatim text refrain versus a historical chapter proximity link, which limits the structural precision of the graph stream.

Finally, the pipeline architecture suffers from tight coupling. Because the text processing and vector steps run sequentially, any adjustment to upstream hyperparameters—such as changing the FastText window size or modifying SVD dimensions—requires recalculating the entire Phase 4 matrix composition pass, creating a development bottleneck during optimization.

VII. Conclusion:

This project demonstrates that a dense, highly descriptive semantic embedding space for Classical Arabic can be trained entirely from internal structural and morphological features without relying on external *tafsir* data. By mapping asymmetric co-occurrence patterns, subword n-grams, derivational templates, and internal cross-reference networks into a unified space via the SharedProjectionFuser, the system balances structural properties with semantic nuance.

Our evaluation results confirm that the system meets scholarly validation benchmarks, achieving a gold standard separation score of 0.542 and a significant root coherence p-value of 0.0021. This proves that ancient, low-resource languages can be modeled directly from their internal textual architectures, providing a robust methodology for commentary-independent semantic search, digital indexing, and computational religious studies.

VIII. **Future Work:**

To build upon these results, several technical updates are recommended for future iterations of the system. First, the rule-based skeleton matcher should be replaced with a machine learning sequence tagger, such as a Conditional Random Field (CRF) trained directly on the complete Leeds dataset, to improve pattern extraction accuracy across weak and mutating roots. Second, the Node2Vec graph engine should be upgraded to a Relational Graph Convolutional Network (R-GCN). This would allow the model to natively process categorical edge attributes, preserving the distinctions between formulaic refrains, historical periods, and root relationships.

Finally, a dedicated vector-caching layer should be implemented at the individual verse level for Phase 4 processing. Decoupling upstream feature generation from the downstream fusion layer would allow researchers to iterate on specific embedding parameters independently, accelerating development cycles.

References:

- [1] *Youtu.be*, 2026. https://youtu.be/u_OyGjmCGqg (accessed Jun. 02, 2026).
- [2] Ahmad-Assaleh, "GitHub - Ahmad-Assaleh/Noori," *GitHub*, 2025. <https://github.com/Ahmad-Assaleh/Noori> (accessed Jun. 02, 2026).
- [3] W. Antoun, F. Baly, and H. Hajj, "AraBERT: Transformer-based Model for Arabic Language Understanding," 2020. Accessed: Jun. 02, 2026. [Online]. Available: <https://aclanthology.org/2020.osact-1.2.pdf>
- [4] M. Mashaabi, S. Al- Khalifa, and H. Al-Khalifa, Eds., "A Survey of Large Language Models for Arabic Language and its Dialects."
- [5] N. V. Otten and N. V. Otten, "Arabic NLP — How To Overcome Challenges, Tutorials In Python & 9 Tools/Resources Including LLMs," *Spot Intelligence*, Oct. 29, 2023. <https://spotintelligence.com/2023/10/29/arabic-nlp/> (accessed Jun. 02, 2026).
- [6] Neri Van Otten and Neri Van Otten, "Top 10 Most Useful Natural Language Processing (NLP) Tools [Libraries & Frameworks] LLMs Included," *Spot Intelligence*, Jan. 29, 2024. <https://spotintelligence.com/2024/01/29/natural-language-processing-nlp-tools/>
- [7] N. V. Otten, "Top 14 Steps To Build A Complete NLTK Preprocessing Pipeline In Python," *Spot Intelligence*, Dec. 21, 2022. <https://spotintelligence.com/2022/12/21/nltk-preprocessing-pipeline/>
- [8] 3Blue1Brown, "But what is a GPT? Visual intro to Transformers | Deep learning, chapter 5," *www.youtube.com*, Apr. 01, 2024. <https://www.youtube.com/watch?v=wjZofJX0v4M>
- [9] "Deep Learning Week 02: Word Embeddings." Accessed: Jun. 02, 2026. [Online]. Available: https://www.cs.cornell.edu/courses/cs4782/2025sp/slides/pdf/week4_2_slides.pdf
- [10] "The Quranic Arabic Corpus - Word by Word Grammar, Syntax and Morphology of the Holy Quran," *corpus.quran.com*. <https://corpus.quran.com/>